

Implementation Plan

PFEVM Health Domain Extension

medical_condition · medication_name

What GLiNER Detects → What the SLM Classifies → What We Generate

This document specifies the exact inputs, outputs, and data flows for integrating SLM-based classification into your FastAPI pipeline.

Component	Status
GLiNER NER	EXISTING — add 2 new labels
SLM Classifier	NEW — fine-tune Qwen2.5-0.5B with LoRA
Paired PHI Resolver	NEW — pure Python logic module
Template Generator	NEW — deterministic lookup from SLM output

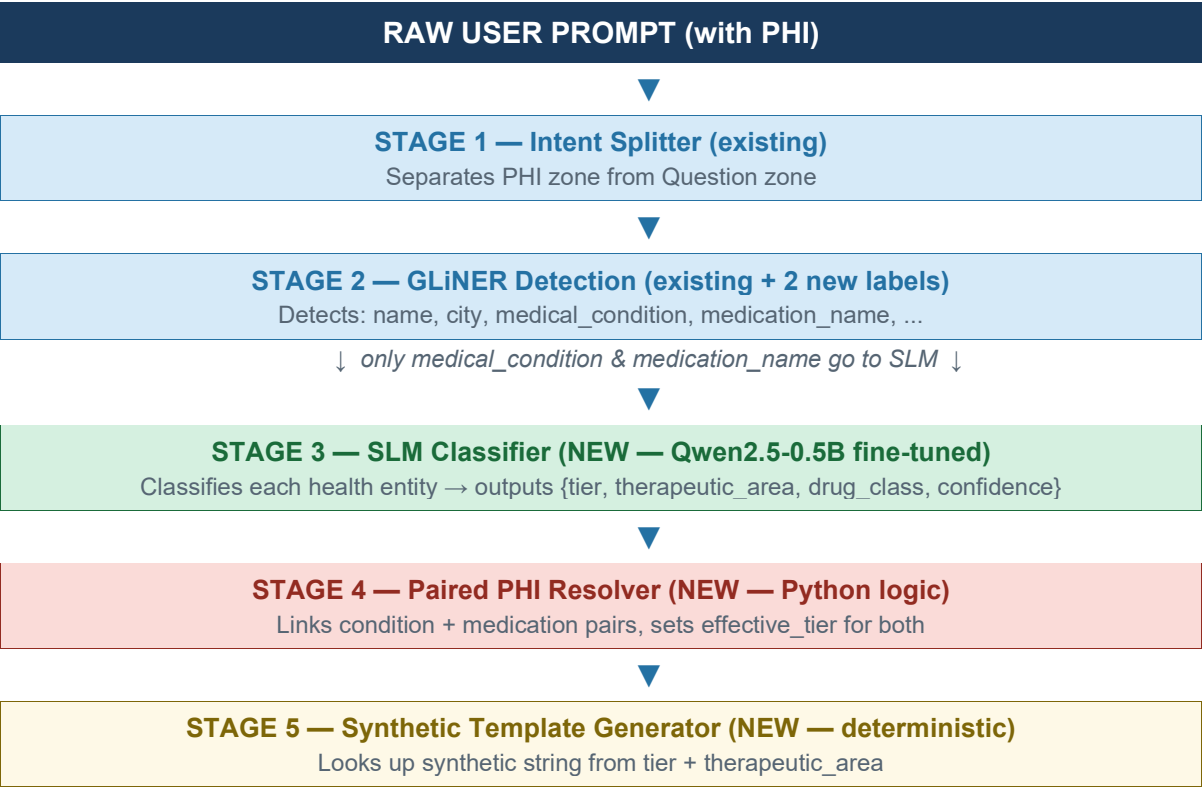
1. End-to-End Data Flow

1.1 The Three Critical Questions

This implementation plan answers three questions:

Stage	Question	Answer (This Document)
1	What does GLiNER detect?	Two entity labels: 'medical_condition' and 'medication_name' added to existing label set. Zero-shot, no retraining needed.
2	What does the SLM receive and output?	Input: entity_text + entity_label + context_window (± 30 tokens). Output: Fixed JSON with tier (1/2/3/null), therapeutic_area, drug_class, confidence.
3	What synthetic data is generated?	Deterministic template lookup: Tier 1 $\rightarrow$ 'mental health condition' / 'antidepressant medication'. Tier 2 $\rightarrow$ 'metabolic condition' / 'diabetes medication'. Tier 3 $\rightarrow$ keep original.

1.2 Full Pipeline — One Prompt, Five Stages





ANONYMIZED PROMPT TO LLM

2. Stage 2 — GLiNER Detection Specification

2.1 What GLiNER Detects

GLiNER (urchade/gliner_medium-v2.1) is a zero-shot NER model. You add new entity labels at inference time — no retraining needed. The existing PFEVM labels remain unchanged. We add exactly two new labels.

Label	Example Detections	Passes to SLM?
name	"Udith"	NO — existing pipeline
city / address	"Kandy", "No. 102, Lake View Road"	NO — existing pipeline
medical_condition	"depression", "Type 2 Diabetes", "asthma"	
medication_name	"Sertraline", "Metformin", "Panadol"	

2.2 Code Change Required

In your existing `universal_redactor.py` file, locate the GLiNER label list and add two strings:

```
# BEFORE (existing PFEVM labels)
labels = [
    "name", "age", "phone number", "email", "address",
    "bank account number", "driver license", "ip address",
    "api key", "credit card number", "city", "country"
]

# AFTER (add two new labels)
labels = [
    "name", "age", "phone number", "email", "address",
    "bank account number", "driver license", "ip address",
    "api key", "credit card number", "city", "country",
    "medical_condition",      # NEW
    "medication_name"        # NEW
]
```

That's it. GLiNER will now detect these entities with no additional configuration.

2.3 GLiNER Output Format (Same as Existing)

GLiNER returns a list of detected entities. Each entity is a dict:

```
{
  "text": "depression",
  "label": "medical_condition",
  "start": 24,
  "end": 34,
  "score": 0.95
}
```

Your existing code already handles this. The only difference: entities with label='medical_condition' or label='medication_name' are routed to the new SLM classifier instead of the existing generators.

3. Stage 3 — SLM Classifier Specification

3.1 What the SLM Receives (Input Schema)

For each detected `medical_condition` or `medication_name` entity, the SLM classifier is called with three inputs extracted from the GLiNER output and surrounding text:

Input Field	Type	Description
<code>entity_text</code>	string	The raw text GLiNER detected. Example: 'Sertraline', 'depression', 'Type 2 Diabetes'
<code>entity_label</code>	string	The GLiNER label: either 'medical_condition' or 'medication_name'
<code>context_window</code>	string	±30 tokens around the entity from the original prompt. Used to disambiguate (e.g., 'Lithium' as drug vs. battery).

Example Input

```
{
  "entity_text": "Sertraline",
  "entity_label": "medication_name",
  "context_window": "I have depression and I am taking Sertraline for that"
}
```

3.2 What the SLM Outputs (Output Schema)

The SLM must output ONLY valid JSON — no explanation, no preamble. The output schema is fixed:

Output Field	Type	Possible Values	Description
<code>tier</code>	int null	1, 2, 3, null	1=Critical (mental health), 2=High (chronic), 3=Standard (OTC), null=not medical
<code>therapeutic_area</code>	string null	See table below	Categorical label: 'mental_health', 'metabolic', 'cardiovascular', etc.
<code>drug_class</code>	string null	See table below	For medication_name only. Example: 'antidepressant', 'antidiabetic'. Null for medical_condition.

confidence	float	0.0 – 1.0	Model's confidence score. If < 0.85, trigger fallback (escalate tier or set tier=1).
------------	-------	-----------	--

Example Output

```
{
  "tier": 1,
  "therapeutic_area": "mental_health",
  "drug_class": "antidepressant",
  "confidence": 0.97
}
```

3.3 All Valid therapeutic_area Values

therapeutic_area	Tier	Examples
mental_health	Tier 1	depression, bipolar, PTSD, schizophrenia
substance_use	Tier 1	alcohol dependency, opioid addiction
metabolic	Tier 2	Type 2 Diabetes, obesity, thyroid disorder
cardiovascular	Tier 2	hypertension, heart failure, stroke
respiratory	Tier 2	asthma, COPD
oncology	Tier 2	breast cancer, lung cancer, leukemia
renal	Tier 2	chronic kidney disease, kidney failure
autoimmune	Tier 2	rheumatoid arthritis, lupus
neurological	Tier 2	epilepsy, Parkinson's, multiple sclerosis
common_acute	Tier 3	cold, flu, headache, fever
null	null	Not a medical entity in this context (e.g., 'Lithium' as battery)

3.4 All Valid drug_class Values (medication_name only)

drug_class	therapeutic_area	Examples
antidepressant	mental_health	Sertraline, Fluoxetine
antipsychotic	mental_health	Risperidone, Olanzapine

mood_stabilizer	mental_health	Lithium, Valproate
anxiolytic	mental_health	Diazepam, Alprazolam
opioid_replacement	substance_use	Methadone, Buprenorphine
addiction_treatment	substance_use	Naltrexone, Disulfiram
antidiabetic	metabolic	Metformin, Insulin
antihypertensive	cardiovascular	Amlodipine, Lisinopril
statin	cardiovascular	Atorvastatin, Simvastatin
bronchodilator	respiratory	Salbutamol
anticancer	oncology	Tamoxifen, Methotrexate
null	common_acute	OTC drugs like Panadol

4. Stage 4 — Paired PHI Resolver Specification

4.1 What It Does

The Paired PHI Resolver links condition + medication pairs detected in the same prompt and ensures they are replaced at the same tier. This prevents medication inference attacks where the medication name reveals the condition even if the condition is anonymized.

The Problem It Solves

User says: 'I have depression and I take Sertraline'

WITHOUT resolver:

depression → SLM outputs tier=1

Sertraline → SLM outputs tier=1

BUT they are replaced independently. No guarantee they match.

WITH resolver:

Detects proximity (28 tokens apart, both in mental_health area)

Creates a PairedPHIUnit with effective_tier = $\min(1,1) = 1$

BOTH are replaced at Tier 1 atomically.

4.2 Input to Resolver

The resolver receives the list of all classified entities from Stage 3 (SLM output). Each entity has:

```
{
  "entity_text": "depression",
  "entity_label": "medical_condition",
  "position": 24, # character offset in prompt
  "tier": 1,
  "therapeutic_area": "mental_health",
  "drug_class": null,
  "confidence": 0.96
}
```

4.3 Pairing Logic (Pseudocode)

```
FUNCTION paired_phi_resolver(classified_entities):

    conditions = [e for e in classified_entities if e.label ==
'medical_condition']
    medications = [e for e in classified_entities if e.label ==
'medication_name']
```

```
paired_units = []
unpaired      = list(classified_entities)

FOR EACH condition IN conditions:
  FOR EACH medication IN medications:

    gap = abs(medication.position - condition.position)

    IF gap <= 150:  # within 150-token proximity window
      IF condition.therapeutic_area == medication.therapeutic_area:

        effective_tier = min(condition.tier, medication.tier)

        paired_units.append({
          'condition':      condition,
          'medication':     medication,
          'effective_tier': effective_tier,
          'therapeutic_area': condition.therapeutic_area
        })

        unpaired.remove(condition)
        unpaired.remove(medication)

RETURN paired_units, unpaired
```

4.4 Output from Resolver

The resolver returns two lists:

- **paired_units**: List of dicts, each containing {condition, medication, effective_tier, therapeutic_area}
- **unpaired**: Entities that did not form pairs — these are replaced individually using their SLM-assigned tier

5. Stage 5 — Synthetic Template Generator Specification

5.1 What It Does

The template generator is deterministic — it receives the tier and therapeutic_area from the SLM (or from the resolver if paired), and looks up the replacement string from a fixed dictionary.

5.2 Replacement Templates

Entity Type	Tier	therapeutic_area	Synthetic Output
medical_condition	1	mental_health	"mental health condition"
medical_condition	1	substance_use	"substance use condition"
medication_name	1	mental_health + drug_class='antidepressant'	"antidepressant medication"
medication_name	1	mental_health + drug_class='antipsychotic'	"antipsychotic medication"
medication_name	1	substance_use + drug_class='opioid_replacement'	"opioid replacement medication"
medical_condition	2	metabolic	"metabolic condition"
medical_condition	2	cardiovascular	"cardiovascular condition"
medical_condition	2	respiratory	"respiratory condition"
medical_condition	2	oncology	"chronic health condition"
medication_name	2	metabolic + drug_class='antidiabetic'	"diabetes medication"
medication_name	2	cardiovascular + drug_class='antihypertensive'	"blood pressure medication"
medication_name	2	cardiovascular + drug_class='statin'	"cholesterol medication"
medication_name	2	respiratory + drug_class='bronchodilator'	"respiratory medication"
ANY	3	common_acute	Keep original entity text unchanged
ANY	null	null (not medical in context)	Do NOT replace — entity is not PHI

5.3 Implementation (Python Dict Lookup)

```
# Tier 1 - Medical Conditions
TIER1_CONDITION_TEMPLATES = {
    "mental_health": "mental health condition",
    "substance_use": "substance use condition",
}

# Tier 1 - Medications
TIER1_MEDICATION_TEMPLATES = {
    "antidepressant": "antidepressant medication",
    "antipsychotic": "antipsychotic medication",
    "mood_stabilizer": "mood stabilizer medication",
    "anxiolytic": "anxiety medication",
    "opioid_replacement": "opioid replacement medication",
    "addiction_treatment": "addiction treatment medication",
}

# Tier 2 - Medical Conditions
TIER2_CONDITION_TEMPLATES = {
    "metabolic": "metabolic condition",
    "cardiovascular": "cardiovascular condition",
    "respiratory": "respiratory condition",
    "oncology": "chronic health condition",
    "renal": "kidney condition",
    "autoimmune": "autoimmune condition",
    "neurological": "neurological condition",
}

# Tier 2 - Medications
TIER2_MEDICATION_TEMPLATES = {
    "antidiabetic": "diabetes medication",
    "antihypertensive": "blood pressure medication",
    "statin": "cholesterol medication",
    "bronchodilator": "respiratory medication",
    "corticosteroid_inhaler": "respiratory medication",
    "anticancer": "cancer treatment medication",
    "immunosuppressant": "immunosuppressant medication",
}

def generate_synthetic(entity_label, tier, therapeutic_area, drug_class,
original_text):
```

```
if tier is None:
    return original_text # not medical – keep as-is

if tier == 3:
    return original_text # OTC – keep as-is

if entity_label == 'medical_condition':
    if tier == 1:
        return TIER1_CONDITION_TEMPLATES[therapeutic_area]
    if tier == 2:
        return TIER2_CONDITION_TEMPLATES[therapeutic_area]

if entity_label == 'medication_name':
    if tier == 1:
        return TIER1_MEDICATION_TEMPLATES[drug_class]
    if tier == 2:
        return TIER2_MEDICATION_TEMPLATES[drug_class]

return '[HEALTH_INFORMATION]' # fallback
```

6. Integration Checklist

Use this checklist to implement the health extension step by step:

1

Extend GLiNER Labels

File: universal_redactor.py or equivalent

Action: Add 'medical_condition' and 'medication_name' to the labels list

Test: Run GLiNER on a test prompt: 'I have depression and take Sertraline'

Expected: GLiNER detects both entities with score > 0.3

Estimated time: 5 minutes

2

Fine-Tune SLM Classifier

Dataset: Use pfevm_health_train.jsonl (from the dataset generator)

Model: Qwen2.5-0.5B-Instruct

Method: LoRA fine-tuning (r=16, alpha=32)

Training: 3-5 epochs on 2,800 training samples

Validation: Check Tier 1 recall ≥ 0.97 on 350 val samples

Output: Save LoRA adapter weights (~40MB file)

Estimated time: 2-3 hours (includes fine-tuning on GPU)

3

Create health_classifier.py Module

Purpose: Wrapper around the fine-tuned SLM

Input: entity_text, entity_label, context_window

Output: {tier, therapeutic_area, drug_class, confidence}

Implementation: Load base Qwen + LoRA adapter at startup

Estimated time: 1 hour

4

Create phi_resolver.py Module

Purpose: Links condition + medication pairs

Input: List of classified entities from Step 3

Output: paired_units[], unpaired[]

Implementation: Pure Python logic — no ML model

Estimated time: 1 hour

5

Create health_generator.py Module

Purpose: Deterministic template lookup

Input: tier, therapeutic_area, drug_class

Output: Replacement string

Implementation: Python dict lookups (see Section 5.3)

Estimated time: 30 minutes

6**Update FastAPI /filter Endpoint**

Add routing logic: if entity.label in ['medical_condition','medication_name'] → call health pipeline

Call sequence: GLiNER → health_classifier → phi_resolver → health_generator

Return: Anonymized text + metadata

Estimated time: 1 hour

7**End-to-End Testing**

Test case 1: 'I have depression and take Sertraline. How to make egg omelet?'

Expected output: 'I have a mental health condition and take antidepressant medication. How to make egg omelet?'

Test case 2: 'I have Type 2 Diabetes and take Metformin'

Expected output: 'I have a metabolic condition and take diabetes medication'

Test case 3: 'I took Panadol for my headache'

Expected output: 'I took Panadol for my headache' (Tier 3 — unchanged)

Estimated time: 1 hour

PFEVM Health Extension — Implementation Plan

GLiNER Detection → SLM Classification → Paired Resolution → Template Generation